

Oznacz jako wykonane

Oznacz to zadanie jako zakończone, kiedy uruchomisz poprawnie kod korzystający z MPI w obu trybach: interaktywnym i wsadowym.

Dostęp do zasobów Cyfronetu

Zaloguj się lub załóż konto na portalu PLGrid. Do poprawnego działania konta wymagana jest afiliacja, która musi być aktywna. Jeżeli wnioskujesz o nową afiliację to przy wypełnianiu wniosku wpisz adres koordynatora przedmiotu (rena@agh.edu.pl).

Prace związane z MPR będą odbywać się w ramach zespołu **plggmpr2025**. Prowadzący jest zobowiązany udostępnić link z zaproszeniem lub rozesłać zaproszenia na adresy email studentów (należy się upomnieć jeżeli tego nie zrobił).

Upewnij się, że w usługach masz aktywną usługę "Access to the Ares cluster at the Cyfronet centre" (Zakładka "Services" -> wyszukaj "Ares" -> Aplikuj).

Po przyjęciu do grupy i zaakceptowaniu aplikacji o usługę, upewnij się, że widoczny jest grant obliczeniowy. Zaloguj się przez **ssh** na **ares.cyfronet.pl** używając swojego loginu i hasła do konta PL-Grid. Następnie wykonaj:

```
$ hpc-grants
```

Na wyświetlonej liście powinien pokazać się grant **plgmpr26** oraz allocation **plgmpr26-cpu**.

Tryb interaktywny

Wykonaj:

```
$ srun --nodes=1 --ntasks=1 --time=00:10:00 --partition=plgrid-testing --account=plgmpr26-cpu --pty /bin/bash
```

W efekcie wykonania powyższego kodu konsola może zawisnąć na czas zależny od obciążenia systemów kolejkowych, po czym powinna uaktywnić się nowa powłoka - katalog roboczy się nie zmieni, ale wszystkie polecenia będą teraz wykonywane w kontekście instancji maszyny obliczeniowej, która właśnie została stworzona - jest to tryb interaktywny korzystania z systemu kolejkowego.

Ten tryb jest bardzo wygodny na etapie tworzenia i testowania rozwiązania. Warto rozumieć parametry polecenia, które go uruchomiło:

- **--nodes** - liczba węzłów obliczeniowych dostępnych dla zadania. Komunikacja pomiędzy nimi może być wolniejsza ze względu na konieczność komunikacji sieciowej, więc na potrzeby zajęć będzie on wynosić zawsze 1.
- **--ntasks** - liczba procesorów dostępnych dla zadania. Komunikacja pomiędzy nimi powinna być szybka, jeżeli są one w obrębie jednego węzła.
- **--time** - czas przydziału instancji w formacie hh:mm:ss, po którym zostanie odebrana. Większy czas daje więcej swobody, jednak może oznaczać dłuższy czas oczekiwania w kolejce.
- **--partition** - różne partycje mają różne parametry, dostępne zasoby i wymagania. "plgrid-testing" jest na ogół dostępny najszybciej, ale ma też najostrejsze ograniczenia. Na potrzeby zajęć będzie zupełnie wystarczający, na potrzeby pracy samodzielnej warto zajrzeć do [dokumentacji](#)
- **--account** - nazwa grantu, w przypadku Aresa musi być dodatkowo ozdobiona suffixem **-cpu**, ponieważ tego typu grant mamy dostępny.
- **--pty /bin/bash** - instrukcja dla polecenia, że powinna uruchomić tryb interaktywny z podaną powłoką.

Domyślnie nie są dostępne żadne narzędzia developerskie - sami musimy ich zażądać. W celu uzyskania dostępu do narzędzi potrzebnych na zajęciach, wywołaj polecenie:

- dla kodu w C: **\$ module add .plgrid plgrid/tools/openmpi**
- dla kodu w Pythonie: **\$ module add scipy-bundle/2021.10-foss-2021b**

Sprawi to, że z poziomu zadania widoczne będą niezbędne narzędzia, które umożliwią budowanie i uruchamianie zadań wykorzystujących MPI. Teraz możesz wykorzystać wiedzę z poprzednich zajęć do uruchomienia swojego kodu - działać będą np. komendy **mpicc** i **mpirun/mpiexec** w przypadku kodu w C. Mogą pojawiać się dodatkowe komunikaty wynikające z mnogości wersji wielu bibliotek, niekoniecznie w pełni ze sobą współpracujących - jeżeli wyniki działania kodu wydają się poprawne, można je zignorować.

W przypadku programu w Pythonie dostępna wersja mpi4py nie jest kompatybilna z uruchamianiem w trybie interaktywnym. W takiej sytuacji konieczne będzie doprowadzenie programu do działania na vClustrze, a następnie wykorzystanie trybu wsadowego do samych pomiarów.

Uruchom tryb interaktywny z parametrem **--ntasks=12**, dodaj moduł MPI, skompiluj kod i porównaj czas wykonania wybranego rozmiaru

problemu dla 1 i 12 procesorów (parametry dla MPI `-np 1` i `-np 12`). Oceń, czy program działa poprawnie - czasy różnią się, widoczne jest przyspieszenie.

Tryb wsadowy

W celu analizy efektywności na potrzeby sprawozdania po następnych zajęciach, potrzebna będzie możliwość uruchomienia kodu bez konieczności nadzoru i utrzymywania aktywnej konsoli. W tym celu potrzebne będzie wykorzystanie komendy `sbatch`. Przyjmuje ona te same parametry, które były analizowane powyżej. Wygodniej jednak jest wykorzystać tę funkcję przekazując do niej skrypt, który będzie zawierać te parametry.

Utwórz nowy plik `<dowolna nazwa>.sh`, który posłuży jako wsad dla systemu kolejkowego. Na początku pliku wstaw shebang oraz parametry dla komendy `sbatch`:

```
#!/bin/bash -l
#SBATCH --nodes 1
#SBATCH --ntasks 12
#SBATCH --time=01:00:00
#SBATCH --partition=plgrid
#SBATCH --account=plgmpr26-cpu
```

Uwaga: wszystkie znaki `"#"` są potrzebne, a pomiędzy `"#"` i `"SBATCH"` nie ma spacji. Jeżeli chcesz zakomentować któryś z parametrów, musisz użyć dwóch znaków `"#"`.

Dalszą część pliku możesz potraktować jak zwykły skrypt bashowy. Umieść w nim komendę, która załaduje moduł MPI (`module add ...`), oraz komendę, która skompiluje program (`mpicc ...`). Na końcu dodaj komendę, która wykona skompilowany program.

Uruchomienie takiego skryptu jest proste:

```
$ sbatch <dowolna nazwa>.sh [argumenty, skryptu...]
```

Otrzymasz informację, że skrypt został zgłoszony do kolejki. Status można podejrzeć komendą `squeue` lub `hpc-jobs`. Zadanie nie jest już zależne od konsoli i rozłączenie/ponowne połączenie się z klastrem nie wpłynie na jego wykonanie. Po zakończeniu działania zadania w katalogu, w którym został on wykonany pojawi się plik `slurm-<nr zadania>.out`, który zawiera całe wyjście zadania. Sprawdź, czy wszystko przebiegło bez problemów.

Jeżeli pomylisz się, możesz wycofać zadanie poleceniem `scancel <nr zadania>`.

Ostatnia modyfikacja: wtorek, 3 marca 2026, 13:54

